User-Defined Primitives (UDP) in Verilog

Digital Design and RTL Fundamentals

October 12, 2026

Contents

1 Introduction **2**

2 Basic UDP Syntax **2**

3 Combinational UDP **2**
3.1 Example: AND Gate UDP 2
3.2 Truth Table 3
3.3 Using the UDP 3

4 UDP Symbols **3**

5 Combinational UDP with Compact Entries **4**
5.1 Example: OR Gate UDP 4

6 Sequential UDP **4**
6.1 Example: Level-Sensitive Storage 4

7 Sequential UDP with Clock Transition **5**
7.1 Example: Positive-Edge-Triggered Storage 5

8 Combinational UDP vs. Sequential UDP **6**

9 UDP Limitations **6**

10 RTL Design Guidelines **6**

11 Summary **7**

1 Introduction

A User-Defined Primitive (UDP) allows a designer to define a logic primitive using a truth table or a state-transition table.

UDP is useful for modeling simple combinational and sequential logic at the primitive level.

Unlike a normal Verilog module, a UDP has exactly one output and one or more inputs.

The two types of UDP are:

- Combinational UDP
- Sequential UDP

UDP definitions use the keywords `primitive`, `table`, and `endprimitive`.

2 Basic UDP Syntax

```
primitive primitive_name (output, input1, input2);

    output output;
    input input1, input2;

    table
        // Input values : Output value
    endtable

endprimitive
```

The output is listed first in the primitive port list. Input ports follow it.

A UDP output must be declared as a scalar `output`. A sequential UDP output is additionally declared as `reg`.

3 Combinational UDP

A combinational UDP defines its output entirely from the current input values.

It does not store previous input or output values.

3.1 Example: AND Gate UDP

```
primitive udp_and (y, a, b);

    output y;
    input a, b;

    table
    // a b : y
       0 0 : 0;
```

```
        0 1 : 0;
        1 0 : 0;
        1 1 : 1;
    endtable

endprimitive
```

3.2 Truth Table

a	b	y
0	0	0
0	1	0
1	0	0
1	1	1

The output is 1 only when both inputs are 1.

3.3 Using the UDP

```
module udp_and_test;

    reg a, b;
    wire y;

    udp_and u1 (y, a, b);

    initial begin
        a = 0; b = 0;
        #10 a = 1;
        #10 b = 1;
        #10 $finish;
    end

    initial
        $monitor("a=%b b=%b y=%b", a, b, y);

endmodule
```

The primitive is instantiated like a module, but its behavior is defined by the UDP table.

4 UDP Symbols

UDP tables support symbols that describe logic values, transitions, and state behavior.

Symbol	Meaning
0	Logic zero
1	Logic one
x	Unknown logic value
?	Any input value: 0, 1, or x
b	Either 0 or 1
-	No change in the output state of a sequential UDP
(r)	Rising transition: 0 to 1
(f)	Falling transition: 1 to 0
(*)	Any change in the input value

Transition symbols are used in sequential UDP tables to describe input changes that trigger state updates.

5 Combinational UDP with Compact Entries

The question mark can simplify a truth table.

5.1 Example: OR Gate UDP

```
primitive udp_or (y, a, b);

    output y;
    input a, b;

    table
    // a b : y
       0 0 : 0;
       1 ? : 1;
       ? 1 : 1;
    endtable

endprimitive
```

The entry `1 ? : 1` means that when `a = 1`, the output is 1 regardless of the value of `b`.

Similarly, `? 1 : 1` covers the cases where `b = 1`.

The table therefore describes the OR function.

6 Sequential UDP

A sequential UDP can store an internal state. Its next state depends on the current input conditions and, where specified, its previous state.

The output of a sequential UDP is declared as both `output` and `reg`.

6.1 Example: Level-Sensitive Storage

```
primitive udp_latch (q, data, enable);

    output q;
    reg q;
    input data, enable;

    table
    // data enable : current_q : next_q
       0     1    :    ?      :    0;
       1     1    :    ?      :    1;
       ?     0    :    ?      :    -;
    endtable

endprimitive
```

Explanation:

- When `enable = 1`, the output follows the data input.
- When `enable = 0`, the output retains its previous state.
- The symbol `-` means no state change.
- The symbol `?` matches any logic value in the corresponding input column.

This example illustrates the behavior of a level-sensitive storage primitive.

7 Sequential UDP with Clock Transition

Sequential UDPs can also describe edge-triggered behavior using transition symbols.

7.1 Example: Positive-Edge-Triggered Storage

```
primitive udp_dff (q, clk, d);

    output q;
    reg q;
    input clk, d;

    table
    // clk  d : current_q : next_q
       (01) 0 :     ?      :    0;
       (01) 1 :     ?      :    1;
    endtable

endprimitive
```

The `(01)` entry specifies a rising edge on the clock input.

At the rising edge, the next state is determined by the value of `d`.

Note: This minimal table illustrates the basic transition behavior. A complete production timing model may require additional entries to describe unknown values and other transitions explicitly.

8 Combinational UDP vs. Sequential UDP

Feature	**Combinational UDP**	**Sequential UDP**
Output behavior	Depends on current inputs	Depends on inputs and stored state
Internal state	Not stored	Stored
Output declaration	`output`	`output reg`
Table format	Inputs and output	Inputs, current state, next state
Typical application	Simple logic gates	Latches and flip-flop behavior

9 UDP Limitations

- A UDP has exactly one scalar output.
- UDP inputs and outputs are single-bit signals.
- A UDP cannot contain internal module instances or procedural `always` blocks.
- A UDP cannot use continuous assignments inside its definition.
- A UDP table defines behavior using primitive-level truth or state-transition entries.
- UDPs are generally used for primitive modeling rather than complex RTL design.

10 RTL Design Guidelines

- Use combinational UDPs for simple single-output logic functions.
- Use sequential UDPs when primitive-level state-transition behavior is required.
- Ensure the output is declared correctly.
- Check all relevant input combinations when writing combinational tables.
- Include suitable transition entries for sequential behavior.
- Verify unknown-value behavior through simulation.
- Prefer ordinary modules and procedural RTL for complex or parameterized designs.
- Check simulator support before using UDPs in a verification flow.

11 Summary

- UDP stands for User-Defined Primitive.
- A combinational UDP defines output values from current input combinations.
- A sequential UDP models state and state transitions.
- The `table` construct specifies primitive behavior.
- UDPs are useful for simple primitive models but are less flexible than modules.

Key takeaway: Use UDPs when a compact, table-based definition of single-bit primitive behavior is appropriate. Use regular Verilog modules for larger and more complex RTL designs.